

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
PROJETO MULTIDISCIPLINAR — TRILHA BACK-END

CINDY PERICO

RU: 0000000

**REDE DE LANCHONETES “RAÍZES DO NORDESTE”:
PROJETO E IMPLEMENTAÇÃO DE UMA API REST MULTICANAL**

CURITIBA

2026

CINDY PERICO

**REDE DE LANCHONETES “RAÍZES DO NORDESTE”:
PROJETO E IMPLEMENTAÇÃO DE UMA API REST MULTICANAL**

Trabalho apresentado à disciplina de Projeto Multidisciplinar — Trilha Back-End do Centro Universitário Internacional UNINTER, como requisito parcial para avaliação da atividade prática.

Prof. Me. Luciane Yanase Kanashiro.

CURITIBA

2026

SUMÁRIO

1 INTRODUÇÃO E OBJETIVOS	5
2 ANÁLISE E REQUISITOS	5
2.1 Requisitos funcionais	5
2.2 Requisitos não funcionais	6
2.3 Regras de negócio e multicanalidade	7
3 DIAGRAMAS	8
3.1 Diagrama de casos de uso	8
3.1.1 Descrição da feature crítica: Realizar Pedido + Solicitar Pagamento	8
3.2 DER — modelo de dados	10
3.3 Arquitetura — camadas e separação de responsabilidades	11
3.4 Diagrama de classes	12
3.5 Diagrama de sequência do fluxo crítico	13
4 API E ENDPOINTS	14
4.1 Convenções do contrato	15
4.2 Endpoints por recurso	16
4.3 Exemplo do fluxo crítico	17
5 LGPD, PRIVACIDADE E SEGURANÇA	18
5.1 Dados pessoais, finalidade e base legal	18
5.2 Consentimento, revogação e anonimização	18
5.3 Controles técnicos	18
6 ENTREGA TÉCNICA	20
6.1 Repositório e evidências	20
6.2 Como executar	20
6.3 MVP entregue	20
7 PLANO DE TESTES	21
7.1 Como reproduzir	21

7.2 Cenários positivos	21
7.3 Cenários negativos	22
7.4 Logs e auditoria	22
7.5 Resultados obtidos	23
8 CONCLUSÃO	24
REFERÊNCIAS	26

1 INTRODUÇÃO E OBJETIVOS

A rede de lanchonetes “Raízes do Nordeste” encontra-se em expansão e passou a operar por múltiplos canais de atendimento — aplicativo, totem de autoatendimento, balcão, retirada (pickup) e web — em unidades distintas, cada uma com cardápio e estoque próprios. Esse crescimento tornou inviável o controle manual e descentralizado das operações, criando a necessidade de um sistema de back-end capaz de consolidar pedidos, estoque, fidelidade e pagamentos de toda a rede.

O objetivo deste trabalho é projetar e implementar a solução de back-end dessa rede, na forma de uma API REST documentada e testável, cobrindo o fluxo crítico do negócio: **criação do pedido, solicitação de pagamento a um gateway externo simulado (mock) e atualização do status do pedido**, com persistência real em banco de dados.

São objetivos específicos: (i) levantar e priorizar os requisitos funcionais e não funcionais do cenário; (ii) modelar o domínio e a base de dados (DER, diagrama de classes e casos de uso); (iii) organizar a solução em camadas com responsabilidades separadas; (iv) definir e documentar o contrato da API em OpenAPI/Swagger; (v) aplicar controles mínimos de segurança e de conformidade com a LGPD; e (vi) evidenciar o funcionamento por meio de um plano de testes reproduzível, com cenários positivos e negativos.

A solução foi implementada em **Java 17** com **Spring Boot 3.3**, **Spring Data JPA**, **Spring Security** com autenticação por token **JWT**, banco **H2** em memória (portável para PostgreSQL por variáveis de ambiente) e documentação automática via **springdoc-openapi**. O código-fonte completo está publicado no repositório público <https://github.com/CindyPerico/trabalhoback-end>.

2 ANÁLISE E REQUISITOS

2.1 Requisitos funcionais

Os requisitos funcionais foram levantados a partir do estudo de caso e priorizados segundo o fluxo crítico do negócio. Todos os requisitos listados no Quadro 1 estão implementados na API entregue.

ID	Requisito funcional	Implementação (endpoint principal)
RF01	Cadastro e autenticação de usuários com perfis/roles	POST /api/auth/registrar, POST /api/auth/login, GET /api/auth/me
RF02	Gestão das unidades da rede	CRUD em /api/unidades
RF03	Consulta do cardápio por unidade, com preço praticado e saldo	GET /api/unidades/{unidadeId}/produtos

ID	Requisito funcional	Implementação (endpoint principal)
RF04	Manutenção do catálogo de produtos e do cardápio da unidade	CRUD em /api/produtos; PUT /api/unidades/{unidadeId}/produtos/{produtoId}
RF05	Realização de pedidos com itens, valores e status	POST /api/pedidos
RF06	Atualização do status do pedido (cozinha, pronto, entregue, cancelado)	PUT /api/pedidos/{id}/status; POST /api/pedidos/{id}/cancelar
RF07	Controle de estoque por unidade (entrada/saída) e restrição de venda	PUT e POST em /api/unidades/{unidadeId}/estoque/produtos/{produtoId}
RF08	Programa de fidelidade com pontos, resgate e consentimento	POST /api/clientes/{id}/pontos e /pontos/resgatar; PUT /api/clientes/{id}/consentimento
RF09	Promoções/campanhas aplicadas ao pedido	POST /api/pedidos/{id}/desconto (regra descrita na seção 2.3)
RF10	Solicitação de pagamento a serviço externo simulado e registro do retorno	POST /api/pedidos/{id}/pagamento/solicitar e /confirmar
RF11	Multicanalidade: registro e filtro do canal de origem do pedido	campo canalPedido em POST /api/pedidos; GET /api/pedidos?canalPedido=
RF12	Conformidade LGPD: consentimento e anonimização do titular	PUT /api/clientes/{id}/consentimento; POST /api/clientes/{id}/anonimizar
RF13	Relatórios gerenciais consolidados e por unidade	GET /api/relatorios/mais-vendidos e /financeiro
RF14	Auditoria de ações sensíveis e de acessos	registro automático em audit_logs e access_logs

Quadro 1 — Requisitos funcionais e endpoints correspondentes.

2.2 Requisitos não funcionais

ID	Requisito não funcional	Como foi atendido
RNF01	Segurança — autenticação	Token JWT assinado (HMAC), stateless, com expiração configurável
RNF02	Segurança — senha	Hash BCrypt; a senha em texto puro nunca é persistida nem retornada
RNF03	Segurança — autorização	Perfis ADMIN, GERENTE e CLIENTE aplicados por rota no SecurityConfig
RNF04	LGPD	Consentimento explícito, anonimização e minimização de dados nas respostas
RNF05	Logs e auditoria	AuditService (ações sensíveis) e AccessAuditFilter (requisições HTTP)
RNF06	Desempenho em horários de pico	Listagens paginadas (page/size), consultas por chave estrangeira e baixa de estoque dentro de transação
RNF07	Tolerância a falhas na integração de pagamento	Pagamento desacoplado do pedido: recusa ou ausência de retorno mantém o pedido em AGUARDANDO_PAGAMENTO, sem baixa de estoque, permitindo nova tentativa
RNF08	Disponibilidade e reprodutibilidade	Aplicação autocontida (H2 em memória + seed), executável com um único comando
RNF09	Documentação	OpenAPI 3 gerado pelo springdoc e publicado no Swagger UI
RNF10	Padronização de erros	Formato JSON único para 400, 401, 403, 404, 409, 422 e 500

Quadro 2 — Requisitos não funcionais.

2.3 Regras de negócio e multicanalidade

O canal de origem é tratado como dado de domínio: o pedido possui o campo obrigatório **canalPedido**, do tipo enumerado, com os valores **APP**, **TOTEM**, **BALCAO**, **PICKUP** e **WEB**. A ausência ou o valor inválido desse campo resulta em erro 400 com a mensagem padronizada, e a listagem aceita o filtro **?canalPedido=**, permitindo consolidar e acompanhar o fluxo de pedidos por canal.

As demais regras do fluxo crítico são:

- o item só pode ser vendido se o produto estiver vinculado ao cardápio da unidade e marcado como disponível;
- produtos sazonais só podem ser vendidos dentro do período configurado;
- o preço unitário é o preço praticado pela unidade (override) ou, na ausência dele, o preço base do catálogo;
- não se cria pedido com estoque insuficiente (conflito 409);
- a baixa de estoque ocorre somente quando o pagamento é aprovado;
- os pontos de fidelidade são creditados automaticamente (1 ponto a cada R\$ 10,00) apenas se o cliente tiver consentimento LGPD ativo;
- promoções são aplicadas como desconto registrado no pedido, com motivo e auditoria, não podendo exceder o subtotal;
- pedidos cancelados ou finalizados não aceitam novas transições de status.

3 DIAGRAMAS

3.1 Diagrama de casos de uso

A Figura 1 apresenta os casos de uso e os atores do sistema: o Cliente (App/Web/Totem), o Atendente (balcão), a Cozinha, o Gerente/Administrador e o sistema externo de pagamento (gateway).

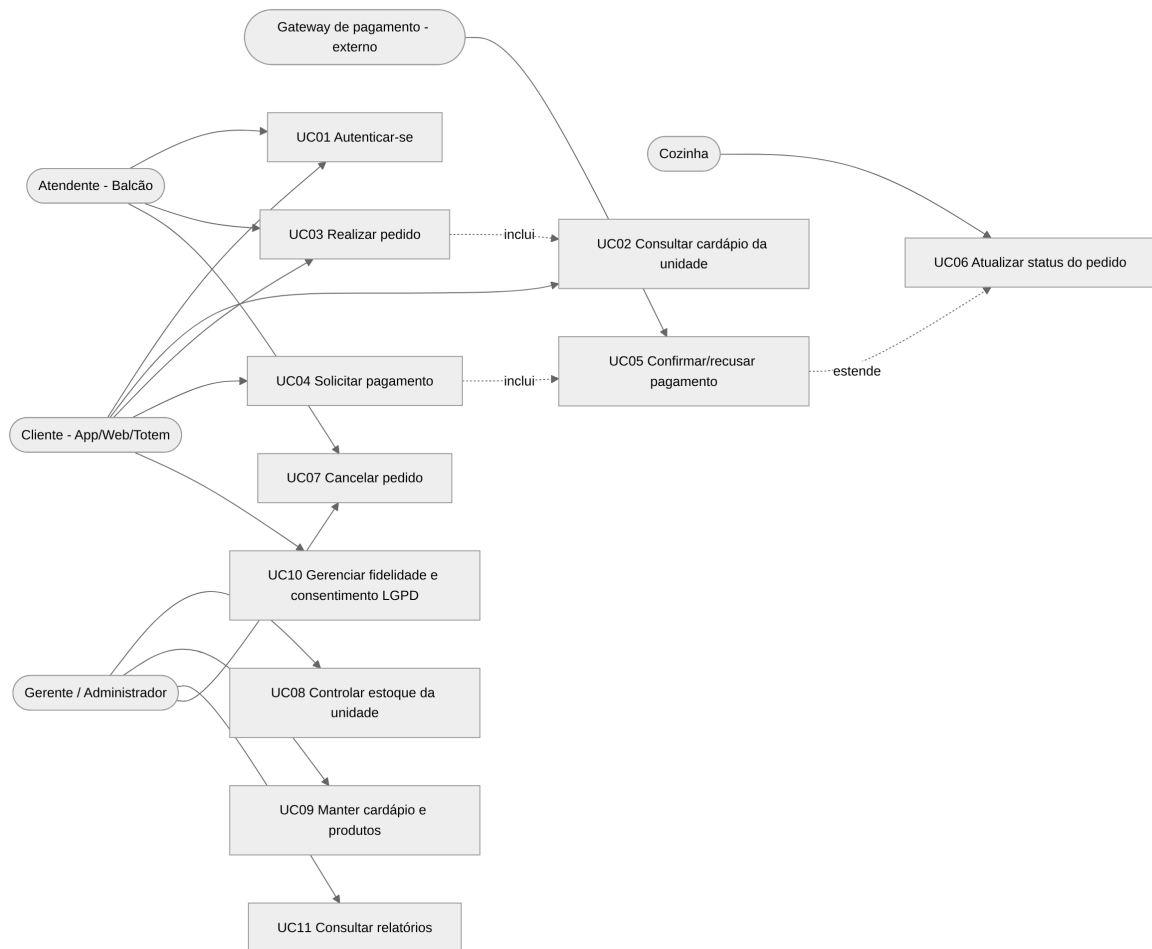


Figura 1 — Diagrama de casos de uso.

3.1.1 Descrição da feature crítica: Realizar Pedido + Solicitar Pagamento

Atores: Cliente (App/Totem/Web) ou Atendente; gateway de pagamento simulado.

Pré-condições: usuário autenticado com token JWT válido; unidade cadastrada; produto vinculado ao cardápio da unidade e disponível; estoque maior ou igual à quantidade pedida.

Fluxo principal:

- o cliente consulta o cardápio da unidade (GET /api/unidades/{unidadeId}/produtos);
- envia POST /api/pedidos informando canalPedido, unidadeId, clienteId (opcional) e itens;

- a API valida cardápio, sazonalidade e estoque, calcula os totais e grava o pedido com status CRIADO;
- o cliente envia POST /api/pedidos/{id}/pagamento/solicitar: o pedido passa a AGUARDANDO_PAGAMENTO e a API gera a referência externa MOCK-<uuid>;
- o gateway devolve o resultado em POST /api/pedidos/{id}/pagamento/confirmar — aprovado: pedido PAGO, baixa de estoque e crédito de pontos;
- a cozinha evolui o status via PUT /api/pedidos/{id}/status (EM_PREPARO, PRONTO, FINALIZADO).

Pós-condições: pedido persistido com itens, totais e histórico em pedido_eventos; estoque decrementado apenas na aprovação; ações sensíveis registradas em audit_logs e requisições em access_logs.

Exceções e regras:

Situação	Tratamento
canalPedido ausente ou inválido	400 Bad Request com o erro padronizado
Unidade, cliente ou produto inexistente	404 Not Found
Produto fora do cardápio da unidade	404 Not Found
Produto indisponível ou fora da sazonalidade	409 Conflict
Estoque insuficiente	409 Conflict — o pedido não é criado
Confirmação antes da solicitação de pagamento	409 Conflict
Pagamento recusado	Pedido permanece AGUARDANDO_PAGAMENTO e o estoque é preservado
Pedido cancelado ou finalizado	Novas transições de status são bloqueadas (409)
Crédito de pontos sem consentimento LGPD	Pontos não são creditados

Quadro 3 — Exceções e regras de negócio do fluxo crítico.

3.2 DER — modelo de dados

A Figura 2 apresenta o Diagrama Entidade-Relacionamento, compatível com o banco utilizado pela API: as tabelas são geradas pelo mapeamento JPA das entidades do pacote *model*. Destacam-se: cada unidade possui estoque próprio (ESTOQUE_ITENS) e cardápio próprio (PRODUTO_UNIDADE); o pedido possui itens (PEDIDO_ITENS) e histórico (PEDIDO_EVENTOS); e o pagamento é desacoplado, registrado no pedido apenas pelo status e pela referência externa.

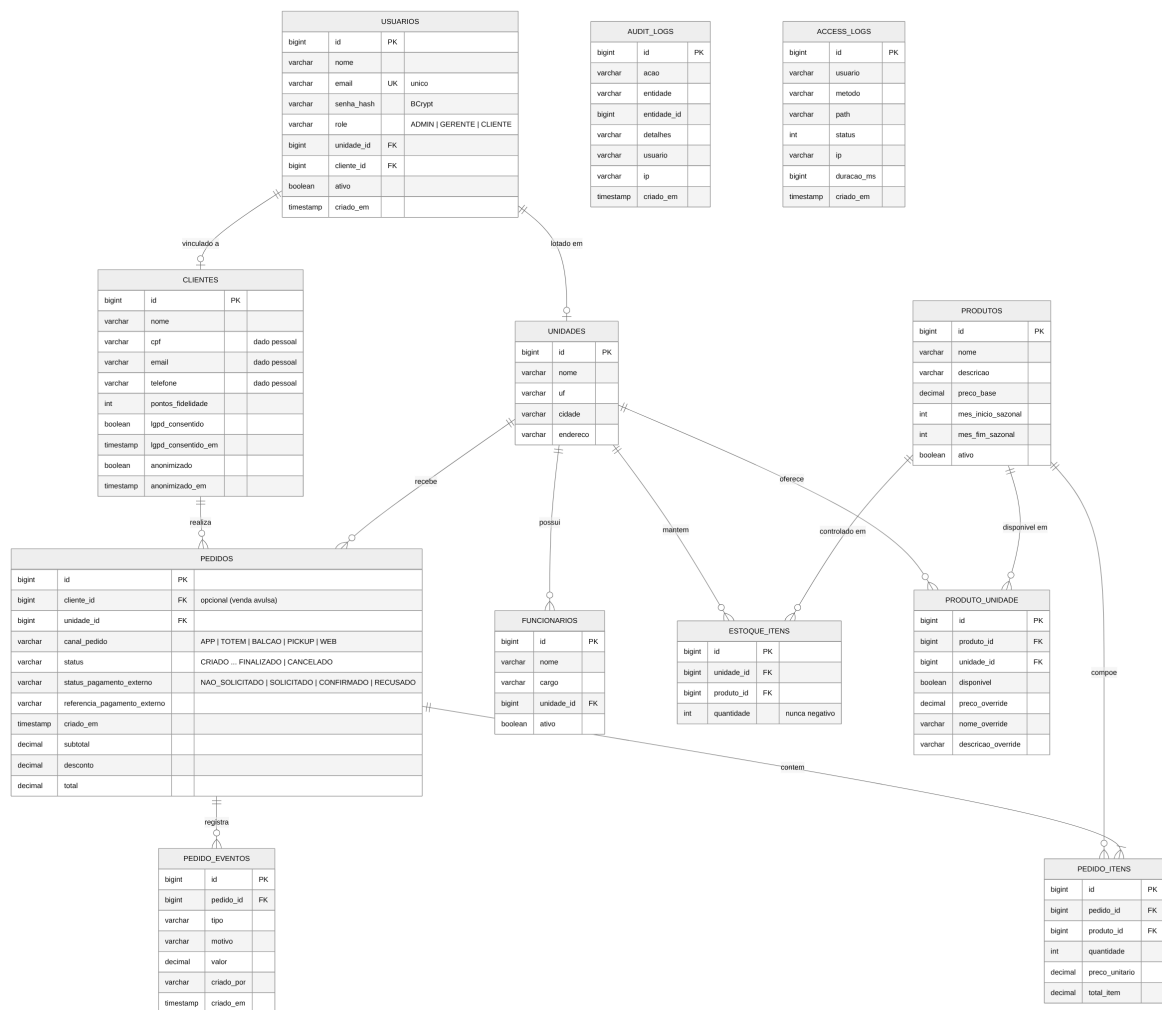


Figura 2 — Diagrama Entidade-Relacionamento (DER).

As principais cardinalidades são: Cliente (1:N) Pedido; Unidade (1:N) Pedido; Pedido (1:N) PedidoItem; Produto (1:N) PedidoItem; Pedido (1:N) PedidoEvento; Unidade (1:N) EstoqueItem; Produto (1:N) EstoqueItem; Unidade (1:N) ProdutoUnidade; Produto (1:N) ProdutoUnidade; Unidade (1:N) Funcionario. O usuário possui vínculos opcionais com unidade (funcionário lotado) e com cliente.

3.3 Arquitetura — camadas e separação de responsabilidades

A solução adota arquitetura em camadas, com dependências orientadas do exterior para o domínio, conforme a Figura 3.

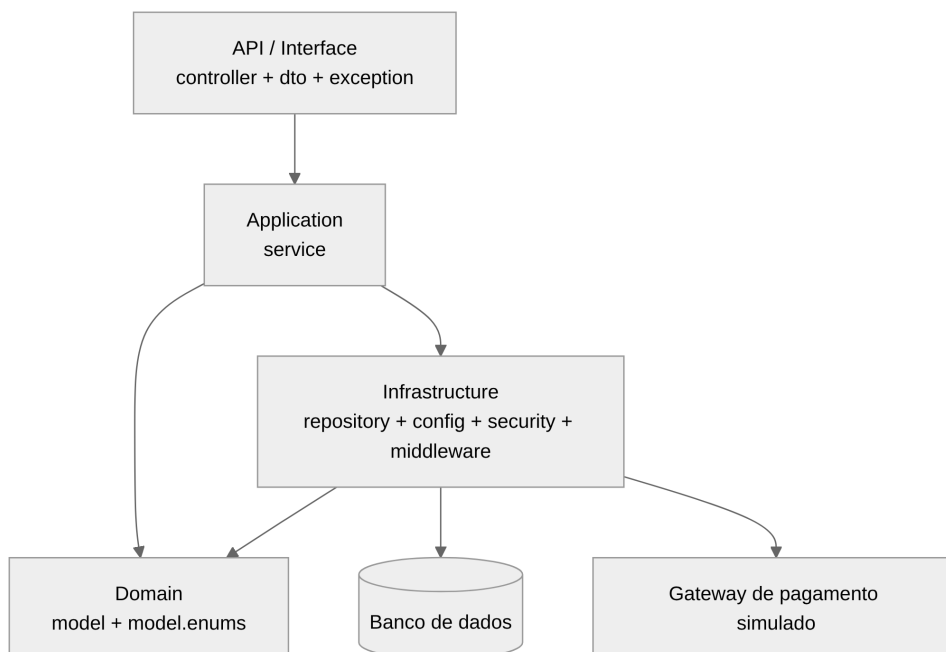


Figura 3 — Camadas da solução.

Camada	Pacote	Responsabilidade
API (Interface)	controller, dto, exception	Rotas REST, contratos de request/response, validação de entrada, documentação OpenAPI e padronização de erros
Application	service	Casos de uso e orquestração: criar pedido, aplicar fidelidade, confirmar pagamento mock, atualizar status e auditar
Domain	model, model.enums	Entidades e estados do negócio (Pedido, Cliente, Produto, EstoqueItem, StatusPedido, CanalAtendimento)
Infrastructure	repository, config, security, middleware	Persistência JPA, carga inicial (seed), autenticação JWT, autorização por perfil e trilha de acessos

Quadro 4 — Camadas, pacotes e responsabilidades.

3.4 Diagrama de classes

A Figura 4 apresenta a visão de domínio, com as principais entidades, seus atributos essenciais e os relacionamentos.

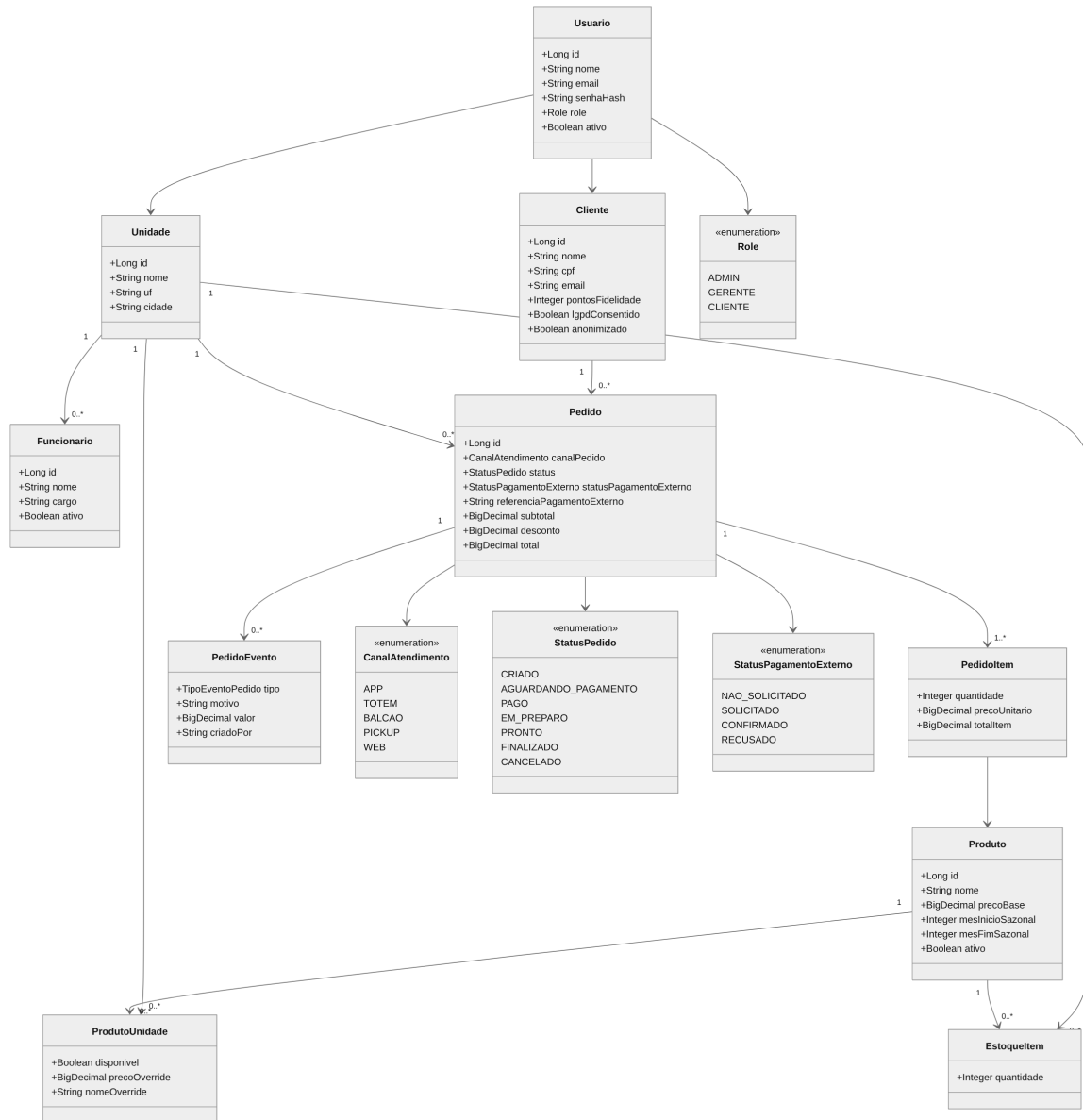


Figura 4 — Diagrama de classes (domínio).

3.5 Diagrama de sequência do fluxo crítico

A Figura 5 detalha a sequência Pedido → Pagamento externo → Atualização de status, incluindo a baixa de estoque e o crédito de pontos após a aprovação.

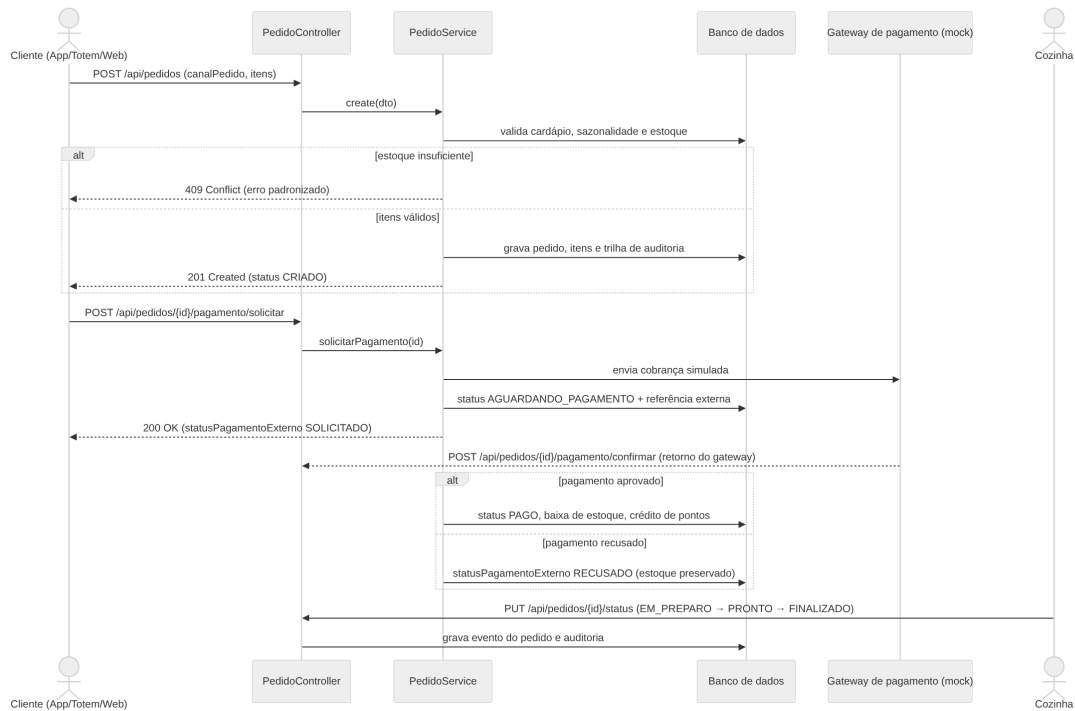


Figura 5 — Diagrama de sequência: pedido, pagamento mock e atualização de status.

A sequência evidencia que a baixa de estoque e o crédito de pontos ocorrem apenas no retorno aprovado do gateway, mantendo o pedido consistente em caso de recusa.

4 API E ENDPOINTS

O contrato da API está publicado em OpenAPI 3 e disponível no Swagger UI da própria aplicação, em **<http://localhost:8082/swagger-ui/index.html>** (documento JSON em **[/v3/api-docs](#)**), refletindo exatamente os endpoints implementados. As Figuras 6 e 7 mostram a documentação gerada.

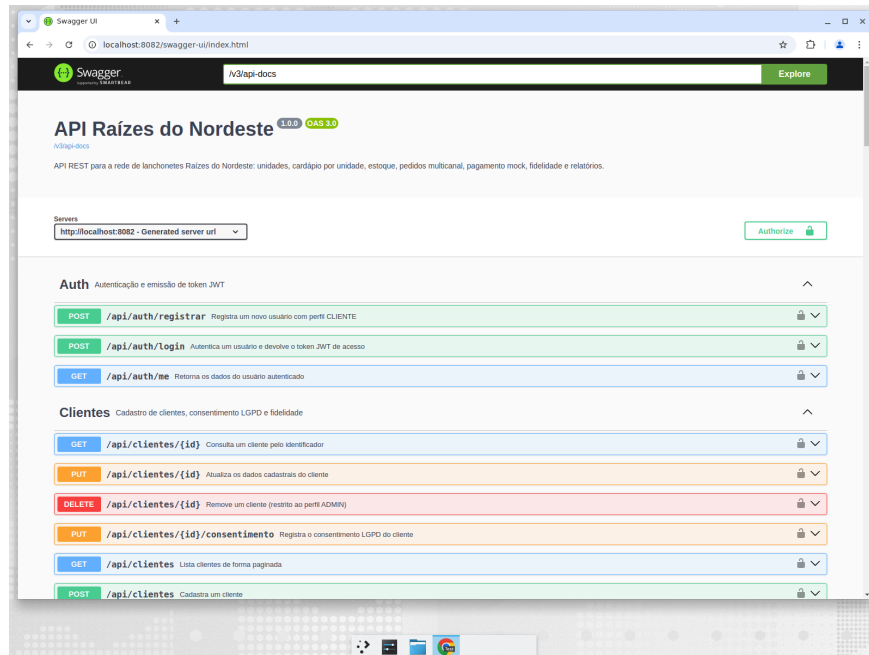


Figura 6 — Swagger UI da API Raízes do Nordeste.

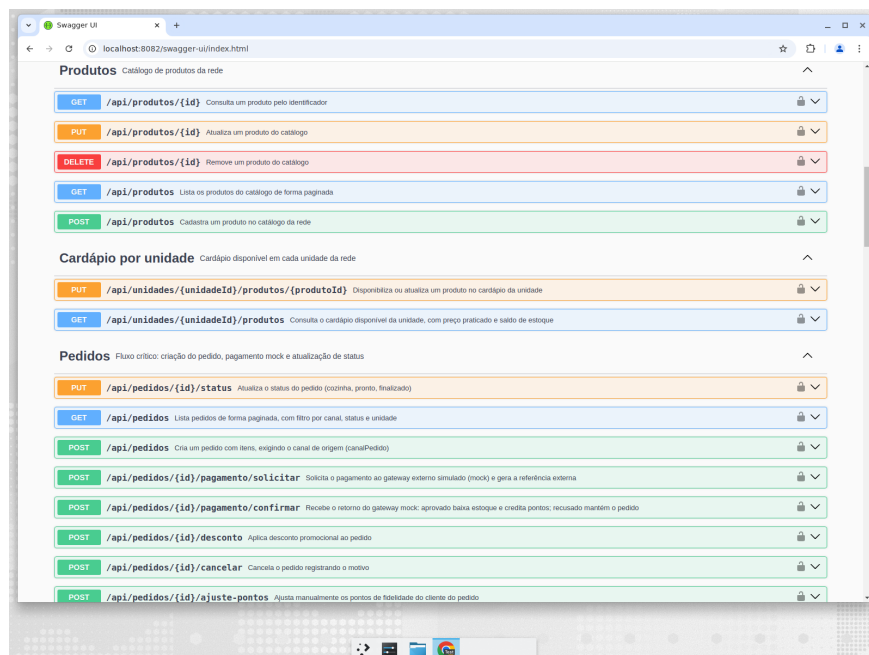


Figura 7 — Recursos de cardápio por unidade e de pedidos no Swagger UI.

4.1 Convenções do contrato

- recursos no plural e identificadores no caminho (ex.: /api/produtos/{id});
- paginação nas listagens por page e size, com resposta contendo content, page, size, totalElements, totalPages e last;
- autenticação por cabeçalho Authorization: Bearer <token>;
- códigos de status coerentes: 200 (ok), 201 (criado), 400/422 (validação), 401 (não autenticado), 403 (sem permissão), 404 (não encontrado) e 409 (conflito);
- corpo de erro sempre no mesmo formato JSON.

Padrão de erro:

```
{
  "timestamp": "2026-08-10T23:15:28.861Z",
  "status": 409,
  "error": "Conflict",
  "message": "Estoque insuficiente",
  "path": "/api/pedidos"
}
```

4.2 Endpoints por recurso

Método e rota	Perfil	Propósito
POST /api/auth/login	público	Autentica e devolve o token JWT de acesso
POST /api/auth/registrar	público	Registra um novo usuário com perfil CLIENTE
GET /api/auth/me	autenticado	Retorna os dados do usuário do token
GET /api/unidades	autenticado	Lista as unidades da rede
GET /api/unidades/{unidadeId}/produtos	autenticado	Consulta o cardápio da unidade com preço praticado e saldo de estoque
PUT /api/unidades/{unidadeId}/produtos/{produtoId}	ADMIN, GERENTE	Disponibiliza ou atualiza um produto no cardápio da unidade
GET /api/produtos	autenticado	Lista o catálogo de forma paginada
GET, PUT, POST /api/unidades/{unidadeId}/estoque/produtos/{produtoId}	ADMIN, GERENTE	Consulta, define e ajusta (entrada/saída) o estoque da unidade
POST /api/pedidos	autenticado	Cria o pedido com itens, exigindo canalPedido
GET /api/pedidos	autenticado	Lista pedidos de forma paginada, com filtros canalPedido, status e unidadeId
GET /api/pedidos/{id}	autenticado	Consulta um pedido com seus itens
PUT /api/pedidos/{id}/status	ADMIN, GERENTE	Atualiza o status do pedido (cozinha, pronto, finalizado)
POST /api/pedidos/{id}/pagamento/solicitar	autenticado	Solicita o pagamento ao gateway simulado e gera a referência externa
POST /api/pedidos/{id}/pagamento/confirmar	autenticado	Recebe o retorno do gateway: aprovado ou recusado
POST /api/pedidos/{id}/desconto	ADMIN, GERENTE	Aplica desconto promocional ao pedido, com motivo
POST /api/pedidos/{id}/cancelar	ADMIN, GERENTE	Cancela o pedido registrando o motivo
POST /api/clientes	ADMIN, GERENTE	Cadastra um cliente
PUT /api/clientes/{id}/consentimento	ADMIN, GERENTE	Registra ou revoga o consentimento LGPD
POST /api/clientes/{id}/pontos e /pontos/resgatar	ADMIN, GERENTE	Credita e resgata pontos de fidelidade
POST /api/clientes/{id}/anonimizar	ADMIN	Anonimiza os dados pessoais do titular
GET /api/relatorios/mais-vendidos e /financeiro	ADMIN, GERENTE	Relatórios consolidados ou por unidade

Quadro 5 — Principais endpoints da API.

A documentação completa de cada endpoint (parâmetros, exemplos de request e response e códigos de status) está no arquivo **docs/ENDPOINTS.md** do repositório e no

Swagger UI.

4.3 Exemplo do fluxo crítico

Requisição — criação de pedido:

```
POST /api/pedidos
Authorization: Bearer <token>

{
  "canalPedido": "TOTEM",
  "clienteId": 1,
  "unidadeId": 1,
  "itens": [ { "produtoId": 1, "quantidade": 2 } ]
}
```

Resposta — 201 Created:

```
{
  "id": 1, "clienteId": 1, "unidadeId": 1,
  "canalPedido": "TOTEM",
  "status": "CRIADO",
  "statusPagamentoExterno": "NAO_SOLICITADO",
  "subtotal": 65.80, "desconto": 0, "total": 65.80
}
```

Resposta — retorno do gateway aprovado (POST /pagamento/confirmar):

```
{
  "id": 1, "canalPedido": "TOTEM",
  "status": "PAGO",
  "statusPagamentoExterno": "CONFIRMADO",
  "referenciaPagamentoExterno": "MOCK-0ae618c6-395e-4398-b179-41c8966a0324",
  "total": 65.80
}
```

5 LGPD, PRIVACIDADE E SEGURANÇA

5.1 Dados pessoais, finalidade e base legal

Dado	Entidade	Finalidade	Base legal (Lei nº 13.709/2018)
Nome	Cliente, Usuario, Funcionario	Identificação no pedido e atendimento	Execução de contrato (art. 7º, V)
CPF	Cliente	Identificação fiscal e do programa de fidelidade	Execução de contrato / obrigação legal (art. 7º, II e V)
E-mail	Cliente, Usuario	Autenticação e comunicação sobre o pedido	Execução de contrato (art. 7º, V)
Telefone	Cliente	Contato operacional sobre o pedido	Execução de contrato (art. 7º, V)
Pontos de fidelidade	Cliente	Programa de benefícios	Consentimento (art. 7º, I)
IP e rota acessada	AccessLog, AuditLog	Segurança da informação e trilha de auditoria	Legítimo interesse (art. 7º, IX)

Quadro 6 — Dados pessoais tratados pela API.

Não são tratados dados sensíveis (art. 5º, II) nem dados de cartão: o pagamento é delegado ao gateway externo simulado e a API armazena apenas a referência da transação.

5.2 Consentimento, revogação e anonimização

O cadastro do cliente inicia sempre com o consentimento em *false*. O consentimento é registrado explicitamente por **PUT /api/clientes/{id}/consentimento**, que grava a flag e o carimbo de data/hora, e é revogado pelo mesmo endpoint. O programa de fidelidade só opera com consentimento ativo: créditos e resgates retornam 409 quando o titular não consentiu, e o crédito automático após o pagamento é ignorado nesse caso. O direito de eliminação (art. 18, VI) é atendido por **POST /api/clientes/{id}/anonimizar**, que remove nome, CPF, e-mail e telefone, preservando os pedidos históricos sem dados identificáveis.

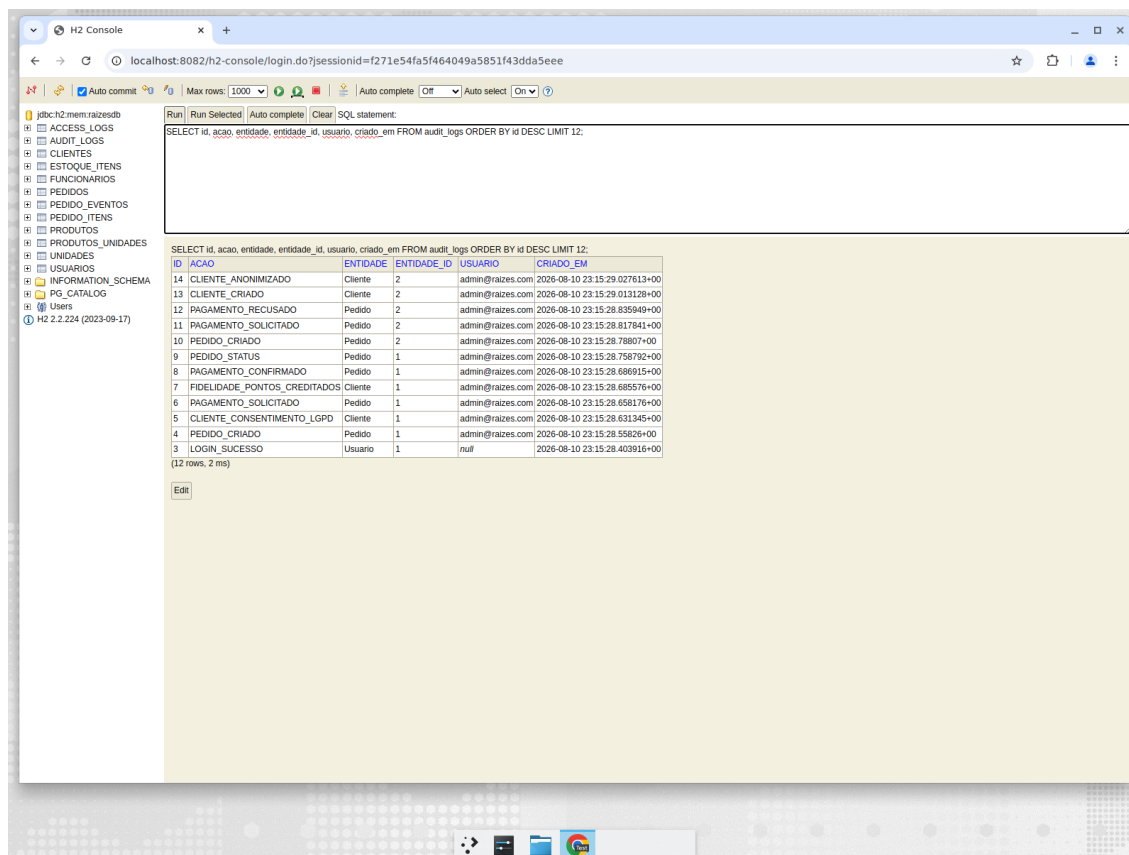
5.3 Controles técnicos

Controle	Implementação
Hash de senha	BCrypt; apenas o hash é persistido em usuarios.senha_hash
Autenticação	Token JWT assinado (HMAC), stateless, com expiração configurável
Autorização	Perfis ADMIN, GERENTE e CLIENTE aplicados por rota
Exposição de dados	Respostas por DTO; a senha nunca é retornada pela API
Logs de acesso	AccessAuditFilter registra usuário, método, rota, status, IP e duração
Auditoria de ações sensíveis	AuditService registra login, criação de pedido, pagamento, alteração de status, consentimento e anonimização

Controle	Implementação
Erros	JSON padronizado, sem stack trace nem detalhes internos
Segredos	JWT_SECRET e credenciais do banco por variáveis de ambiente (.env.example), fora do código-fonte

Quadro 7 — Controles técnicos de segurança e privacidade.

A Figura 8 evidencia a trilha de auditoria gravada em banco durante a execução do fluxo crítico, com o registro de login, criação de pedido, pagamento, alteração de status, consentimento e anonimização.



The screenshot shows the H2 Console interface with a SQL query executed against the audit_logs table. The query is: `SELECT id, acao, entidade, entidade_id, usuario, criado_em FROM audit_logs ORDER BY id DESC LIMIT 12;`. The results are displayed in a table with 12 rows and 6 columns: ID, AÇÃO, ENTIDADE, ENTIDADE_ID, USUARIO, and CRIADO_EM.

ID	AÇÃO	ENTIDADE	ENTIDADE_ID	USUARIO	CRIADO_EM
14	CLIENTE_ANONIMIZADO	Cliente	2	admin@raizes.com	2026-08-10 23:15:29.027613+00
13	CLIENTE_CRIADO	Cliente	2	admin@raizes.com	2026-08-10 23:15:29.013128+00
12	PAGAMENTO_RECUSADO	Pedido	2	admin@raizes.com	2026-08-10 23:15:28.835949+00
11	PAGAMENTO_SOLICITADO	Pedido	2	admin@raizes.com	2026-08-10 23:15:28.817841+00
10	PEDIDO_CRIADO	Pedido	2	admin@raizes.com	2026-08-10 23:15:28.78807+00
9	PEDIDO_STATUS	Pedido	1	admin@raizes.com	2026-08-10 23:15:28.758792+00
8	PAGAMENTO_CONFIRMADO	Pedido	1	admin@raizes.com	2026-08-10 23:15:28.686915+00
7	FIDELIDADE_PONTOS_CREDITADOS	Cliente	1	admin@raizes.com	2026-08-10 23:15:28.685576+00
6	PAGAMENTO_SOLICITADO	Pedido	1	admin@raizes.com	2026-08-10 23:15:28.658176+00
5	CLIENTE_CONSENTIMENTO_LGPD	Cliente	1	admin@raizes.com	2026-08-10 23:15:28.631345+00
4	PEDIDO_CRIADO	Pedido	1	admin@raizes.com	2026-08-10 23:15:28.55826+00
3	LOGIN_SUCESSO	Usuario	1	null	2026-08-10 23:15:28.403816+00

(12 rows, 2 ms)

Figura 8 — Registros da tabela audit_logs após a execução do fluxo crítico.

6 ENTREGA TÉCNICA

6.1 Repositório e evidências

- Repositório público: <https://github.com/CindyPerico/trabalhoback-end>
- Swagger UI (local): <http://localhost:8082/swagger-ui/index.html> — documento OpenAPI em **/v3/api-docs**
- Coleção Postman: **docs/postman/RaizesDoNordeste.postman_collection.json** e environment **docs/postman/RaizesDoNordeste.postman_environment.json**
- Plano de testes: **TEST_PLAN.md**
- DER, casos de uso e classes: **docs/DIAGRAMAS/** (fontes) e **docs/imagens/** (imagens)
- Saída completa da execução dos cenários: **docs/evidencias-execucao.txt**

6.2 Como executar

Requisitos: Java 17 (JDK) e Maven 3.9. O banco é H2 em memória, portanto não é necessário instalar SGBD; as tabelas são criadas na inicialização e a carga inicial (seed) cria 2 unidades, 3 produtos com estoque, 1 cliente e 3 usuários.

```
git clone https://github.com/CindyPerico/trabalhoback-end.git
cd trabalhoback-end
cp .env.example .env          # opcional: ajusta porta, banco e JWT_SECRET
mvn clean package             # instala dependências e compila
mvn spring-boot:run           # inicia a API em http://localhost:8082
mvn test                       # executa os 17 testes automatizados
```

Usuários criados pelo seed para teste: **admin@raizes.com / admin12345** (ADMIN), **gerente@raizes.com / gerente12345** (GERENTE) e **maria.souza@example.com / cliente12345** (CLIENTE).

6.3 MVP entregue

O fluxo obrigatório escolhido foi o **Fluxo A — Pedido → Pagamento mock → Atualização de status**, entregue completo e testável, com persistência real (CRUD em banco), autenticação JWT com perfis e padrão de erro consistente. O Fluxo B (estoque por unidade) também está implementado, já que a baixa e o controle de saldo por unidade fazem parte das regras do fluxo principal.

7 PLANO DE TESTES

7.1 Como reproduzir

A validação é reproduzível por dois caminhos complementares. O primeiro são os **17 testes automatizados** (JUnit 5 + MockMvc), executados por **mvn test**, que sobem o contexto completo da aplicação com banco e seed próprios. O segundo é a **coleção Postman**, com 22 requisições organizadas nas pastas Auth, Cardápio, Pedidos, Pagamento, Fidelidade e LGPD, Relatorios e Erros, cada uma com asserções automáticas (pm.test). A ordem sugerida de execução é a das pastas; o login salva automaticamente as variáveis de ambiente token, tokenCliente e pedidoId. Pela linha de comando:

```
newman run docs/postman/RaizesDoNordeste.postman_collection.json \
-e docs/postman/RaizesDoNordeste.postman_environment.json
```

Pré-condição comum a todos os cenários: aplicação em execução com o seed aplicado.

7.2 Cenários positivos

ID	Cenário / Endpoint	Entrada	Esperado	Evidência
T01	Login válido — POST /api/auth/login	{"email":"admin@raizes.com","senha":"admin12345"}	200 + accessToken e role ADMIN	Postman Auth/T01; AutenticacaoTest.loginValido
T02	Usuário autenticado — GET /api/auth/me	token válido	200 + e-mail do token	Postman Auth/T03
T03	Cardápio por unidade — GET /api/unidades/1/produtos	unidadeId=1	200 + itens com preço e quantidadeEmEstoque	Postman Cardapio/T04; PedidoFluxoTest.cardapioPorUnidade
T04	Criar pedido — POST /api/pedidos	{"canalPedido":"TOTE M","clienteId":1,"unidadeId":1,"itens":[{"produtoId":1,"quantidade":2}]}	201 + canalPedido TOTEM, status CRIADO e total calculado	Postman Pedidos/T06; PedidoFluxoTest.fluxoPedidoPagamentoStatus
T05	Filtrar por canal — GET /api/pedidos?canalPedido=TOTEM	canalPedido=TOTEM &page=0&size=10	200 + página só com pedidos TOTEM	Postman Pedidos/T07; PedidoFluxoTest.filtrarPorCanal
T06	Pagamento aprovado — POST /pagamento/solicitar e /confirmar	{"confirmado":true}	200 + status PAGO, pagamento CONFIRMADO e estoque decrementado de 50 para 48	Postman Pagamento/T08 e T09; PedidoFluxoTest.fluxoPedidoPagamentoStatus
T07	Atualizar status — PUT /api/pedidos/{id}/status	{"status":"EM_PREPARO"}	200 + status EM_PREPARO	Postman Pagamento/T10

ID	Cenário / Endpoint	Entrada	Esperado	Evidência
T08	Fidelidade com consentimento — PUT /consentimento e POST /pontos	{"consentido":true}	200 + lgpdConsentido true e saldo de pontos atualizado	Postman Fidelidade e LGPD/T12 e T13; FidelidadeLgpdTest
T09	Relatório financeiro — GET /api/relatorios/financeiro?unidadeId=1	unidadeId=1	200 + totalPedidos e totalVendido	Postman Relatorios/T16

Quadro 8 — Cenários de teste positivos.

7.3 Cenários negativos

ID	Cenário / Endpoint	Entrada	Esperado	Evidência
T10	Acesso sem token — GET /api/pedidos	sem Authorization	401 + erro padronizado	Postman Erros/T17; AutenticacaoTest.semToken
T11	Perfil sem permissão — GET /api/relatorios/financeiro	token de CLIENTE	403 + “Acesso negado”	Postman Erros/T18; AutenticacaoTest.clienteSemPermissao
T12	Credenciais inválidas — POST /api/auth/login	senha incorreta	401	AutenticacaoTest.loginInvalido
T13	Campo obrigatório ausente — POST /api/pedidos	body sem canalPedido	400 + mensagem citando canalPedido	Postman Erros/T19; PedidoFluxoTest.pedidoSemCanal
T14	Formato/valor inválido — POST /api/pedidos e /api/clientes	quantidade -3; e-mail inválido	400 + erro de validação	Postman Erros/T22; FidelidadeLgpdTest.emailInvalido
T15	Produto inexistente — POST /api/pedidos	produtoId 9999	404	Postman Erros/T20; PedidoFluxoTest.pedidoProdutoInexistente
T16	Estoque insuficiente — POST /api/pedidos	quantidade 9999	409 + “Estoque insuficiente”	Postman Erros/T21; PedidoFluxoTest.pedidoEstoqueInsuficiente
T17	Pagamento recusado — POST /pagamento/confirmar	{"confirmado":false}	200 + pagamento RECUSADO, pedido segue AGUARDANDO _PAGAMENTO e estoque intacto	Postman Pagamento/T11; PedidoFluxoTest.pagamentoRecusado
T18	Fidelidade sem consentimento e resgate sem saldo — POST /pontos e /pontos/resgatar	{"pontos":9999}	409 nos dois casos	FidelidadeLgpdTest.fidelidadeComConsentimento

Quadro 9 — Cenários de teste negativos.

7.4 Logs e auditoria

ID	Cenário	Verificação	Evidência
T19	Ação sensível gera registro de auditoria	A criação de pedido grava PEDIDO_CRIADO em audit_logs e toda requisição gera registro em access_logs	PedidoFluxoTest.auditoriaDoPedido; Figura 8 (consulta no H2 Console)

Quadro 10 — Cenário de auditoria.

7.5 Resultados obtidos

São 19 cenários documentados — 9 positivos, 9 negativos e 1 de auditoria —, acima do mínimo exigido (10 cenários, com 6 positivos e 4 negativos). Todos foram executados contra a API em funcionamento; a saída completa está em **docs/evidencias-execucao.txt**. A suíte automatizada foi executada com sucesso:

```
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 -- PedidoFluxoTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- FidelidadeLgpdTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- AutenticacaoTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- ApplicationTests
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Exemplos de respostas obtidas na execução dos cenários negativos, todas no formato de erro padronizado:

```
T13 400 {"status":400,"error":"Bad Request",
        "message":"canalPedido: must not be null","path":"/api/pedidos"}
T15 404 {"status":404,"error":"Not Found",
        "message":"Produto nao encontrado","path":"/api/pedidos"}
T16 409 {"status":409,"error":"Conflict",
        "message":"Estoque insuficiente","path":"/api/pedidos"}
T10 401 {"status":401,"error":"Unauthorized",
        "message":"Autenticacao necessaria: informe um token JWT valido"}
T11 403 {"status":403,"error":"Forbidden",
        "message":"Acesso negado: perfil sem permissao para este recurso"}
```

8 CONCLUSÃO

O trabalho entregou uma API REST funcional, documentada e reproduzível para a rede “Raízes do Nordeste”, com o fluxo crítico fechado de ponta a ponta: criação do pedido com validação de cardápio, sazonalidade e estoque; solicitação de pagamento ao gateway externo simulado; recebimento do retorno; e atualização do status do pedido, com persistência real em banco.

A priorização seguiu o MVP obrigatório. Primeiro foi implementado o fluxo Pedido → Pagamento mock → Status, por ser o que sustenta a operação da rede e o que atravessa todas as entidades do domínio. Em seguida vieram os controles transversais — autenticação JWT, autorização por perfil, padronização de erros e auditoria — que são requisitos não funcionais sem os quais a solução não se sustentaria em produção. Por último, os cadastros de apoio e os relatórios gerenciais.

Os artefatos de modelagem conectam-se diretamente ao código entregue. As entidades do DER correspondem às tabelas geradas pelo mapeamento JPA das classes do pacote *model*; o diagrama de classes reflete esse mesmo domínio e seus enumerados (CanalAtendimento, StatusPedido, StatusPagamentoExterno); e cada caso de uso possui endpoint correspondente — UC02 em GET /api/unidades/{id}/produtos, UC03 em POST /api/pedidos, UC04 e UC05 no par pagamento/solicitar e pagamento/confirmar, UC06 em PUT /api/pedidos/{id}/status e UC10 nos endpoints de fidelidade e consentimento.

O pagamento foi tratado como integração simulada e desacoplada: a solicitação gera a referência MOCK-<uuid> e coloca o pedido em AGUARDANDO_PAGAMENTO; o retorno do gateway é recebido pelo endpoint de confirmação e, quando aprovado, o pedido passa a PAGO, o estoque é baixado e os pontos de fidelidade são creditados; quando recusado, o estoque permanece intacto e uma nova tentativa é possível. Esse desacoplamento é também a estratégia de tolerância a falhas na integração. As validações e erros foram padronizados em um único formato JSON, com 400 e 422 para validação, 401 para não autenticado, 403 para perfil sem permissão, 404 para recurso inexistente e 409 para conflito de regra de negócio.

Em segurança e privacidade, os principais cuidados foram o armazenamento de senhas apenas como hash BCrypt, a autenticação stateless por token JWT com expiração, a autorização por perfil em cada rota, a exposição de dados por DTO, o consentimento explícito como condição para o programa de fidelidade, a anonimização do titular e a trilha de auditoria de ações sensíveis e de acessos. Os testes evidenciam esse funcionamento: 17 testes automatizados e 19 cenários documentados cobrem autenticação e autorização, validação de dados, regras do fluxo principal, pagamento aprovado e recusado e o registro em auditoria.

Ficaram como proposta de evolução: campanhas promocionais automáticas por regra (hoje o desconto é aplicado manualmente ao pedido, com motivo e auditoria), *refresh token* e revogação de credenciais, migrations versionadas para ambientes de produção com PostgreSQL, idempotência por chave na criação de pedido e notificação assíncrona do status ao cliente.

Respondendo à reflexão final proposta pelo roteiro, a solução se sustentaria em uma entrevista técnica: entrega um fluxo de negócio completo em vez de um conjunto disperso de endpoints, apresenta separação clara de responsabilidades, contrato documentado, tratamento consistente de erros, controles de segurança e privacidade e evidências de teste reproduzíveis por qualquer avaliador.

REFERÊNCIAS

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures.** 2000. Tese (Doutorado em Ciência da Computação) — University of California, Irvine, 2000.

FOWLER, Martin. **Patterns of enterprise application architecture.** Boston: Addison-Wesley, 2003.

H2 DATABASE ENGINE. **H2 database documentation.** Disponível em: <https://www.h2database.com/html/main.html>. Acesso em: 10 ago. 2026.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **RFC 7519: JSON Web Token (JWT).** Internet Engineering Task Force, 2015. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Acesso em: 10 ago. 2026.

OPENAPI INITIATIVE. **OpenAPI specification.** Disponível em: <https://spec.openapis.org/oas/latest.html>. Acesso em: 10 ago. 2026.

RICHARDSON, Leonard; AMUNDSEN, Mike. **RESTful web APIs.** Sebastopol: O'Reilly Media, 2013.

SPRING. **Spring Boot reference documentation.** Disponível em: <https://docs.spring.io/spring-boot/documentation.html>. Acesso em: 10 ago. 2026.

SPRING. **Spring Security reference documentation.** Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 10 ago. 2026.